Project: Socket programming

This document provides a comprehensive guide to building a basic client-server application in Java using sockets. It covers the underlying principles of socket programming, the specific classes used from the Java standard library, and the detailed flow of execution. The example demonstrates a single-class implementation for simplicity, where both client and server logic reside within the same class, using threads for concurrent execution. While not a production-ready server, it serves as an excellent foundation for learning network programming in Java.

Key Components and Concepts:

- **Socket Programming:**
    - **Definition:** Socket programming is a fundamental way for programs to communicate over a network. A socket acts like an endpoint for communication, much like a telephone jack. Two programs, a *client* and a *server*, establish a connection through their respective sockets.
    - **Client-Server Model:** This is a common architectural pattern. The *server* listens for incoming connections on a known *port*. The *client* initiates a connection to the server's address and port. Once the connection is established, both client and server can send and receive data.
    - **TCP/IP:** This example uses TCP (Transmission Control Protocol), a reliable, connection-oriented protocol. TCP ensures that data is delivered in order and without errors. TCP operates on top of IP (Internet Protocol), which handles the routing of data packets across the network.

- `ServerSocket` **(Server-Side):**
    - **Purpose:** The `ServerSocket` class is specifically designed for the *server* side of a connection. Its primary role is to *listen* for incoming client connection requests.
    - **Binding:** When you create a `ServerSocket`, you *bind* it to a specific port on the server machine. This means the server will monitor that port for incoming connections.
    - **`accept()` Method:** The most important method of `ServerSocket`. It *blocks* (pauses execution) until a client attempts to connect to the server's port. When a connection request arrives, `accept()` does the following:

        - Accepts the connection.
        - Creates a *new* `Socket` object. This new `Socket` represents the *established connection* between the server and that particular client.
        - Returns the new `Socket` object. The server then uses this new `Socket` to communicate with the connected client. The original `ServerSocket` continues listening for *other* client connections.

- **`Socket` (Client-Side and Server-Side):**
    - **Client-Side:** The client uses a `Socket` to *initiate* the connection to the server.
        - **Constructor:** The client creates a `Socket` object, typically providing the server's hostname (or IP address) and port number. This attempts to establish the connection.
        - **Example:** `new Socket("localhost", 12345);` // Connects to the server on the same machine (localhost) at port 12345.
    - **Server-Side:** As explained above, the `ServerSocket.accept()` method returns a *new* `Socket` object for each successful client connection. This `Socket` is *different* from the `ServerSocket`. It represents the dedicated communication channel with that one client.

- **`PORT`:**
    - **Definition:** A port is a 16-bit number (0-65535) that acts like a logical endpoint for communication on a network. Think of an IP address as the street address of a building, and the port number as the apartment number within that building.
    - **Well-Known Ports:** Ports 0-1023 are considered "well-known" ports and are typically reserved for standard services (e.g., HTTP on port 80, HTTPS on port 443, SSH on port 22).
    - **Ephemeral Ports:** When a client connects to a server, the client's operating system automatically assigns the client an unused port number (usually from a higher range). This is called an ephemeral port.

- **Choosing a Port:** For your own applications (especially during development), you should choose a port number above 1023 that is not already in use by another application. Common choices are in the range of 8000, 8080, 12345, etc.

- **`InputStream` and `OutputStream`:**

  - **Streams:** In Java, streams represent sequences of data. They are used for reading and writing data in a sequential manner.
  - **`InputStream`:** Used for *reading* data. You read bytes *from* an `InputStream`.
  - **`OutputStream`:** Used for *writing* data. You write bytes *to* an `OutputStream`.
  - **Socket Streams:** `Socket` objects provide both an `InputStream` (to receive data from the other end of the connection) and an `OutputStream` (to send data to the other end). You obtain these streams using `socket.getInputStream()` and `socket.getOutputStream()`.

- **`InputStreamReader`:**

  - **Purpose:** To bridge the gap between byte streams and character streams. It is used to read bytes and decodes them into characters using a specified charset.
  - **Usage:** Wraps an `InputStream` (like the one from a `Socket`) to allow reading character data.

- **`BufferedReader`:**

  - **Purpose:** To improve the efficiency of reading text. Reading directly from a raw `InputStream` can be slow because it might involve many small, individual read operations from the underlying network connection.
  - **Construction:** You create a `BufferedReader` by wrapping it around an existing `Reader` (which itself usually wraps an `InputStream`).
    - `socket.getInputStream()`: Gets the raw byte stream from the socket.

        - `new InputStreamReader(...)`: Wraps the `InputStream` and converts bytes to characters (using a default character encoding, or you can specify one).
        - `new BufferedReader(...)`: Wraps the `InputStreamReader` to add buffering and the `readLine()` method.

- **`PrintWriter`:**
    - **Purpose:** To provide convenient methods for writing formatted text to an `OutputStream`.
    - **`println()` Method:** Similar to `System.out.println()`, but it writes to the associated output stream (and thus, over the network in the case of a socket).
    - **Auto-Flushing:** When you create a `PrintWriter` with the `autoFlush` parameter set to `true`, the output buffer is automatically flushed (sent) after each `println()` call. This ensures that the data is sent immediately, which is important for interactive communication.
    - **Construction:** You create a `PrintWriter` by wrapping it around an existing `OutputStream` (or a `Writer`). You can also specify whether to enable auto-flushing.
        - `socket.getOutputStream()`: Gets the raw byte stream from the socket.
        - `new PrintWriter(..., true)`: Creates a `PrintWriter` that wraps the `OutputStream` and enables auto-flushing.
        - `new PrintWriter(...)` : Creates a `PrintWriter` that wraps the `OutputStream` without enabling auto-flushing.

- **`Thread`:**
    - **Concurrency:** Threads allow a program to perform multiple tasks concurrently (seemingly at the same time). This is essential for servers that need to handle multiple clients simultaneously.
    - **`extends Thread`:** One way to create a thread is to create a class that *extends* the `Thread` class.

- **`run()` Method:** The code that you want to execute in a separate thread is placed in the `run()` method. When you call `thread.start()`, a new thread is created, and the `run()` method is executed in that new thread.
- **`start()` Method:** The correct method to start execution of the `run` method of a thread.

- **Logger:**
  - **Purpose**: Provides a standardized way to record events, messages, and errors during the execution of a program.
  - **Usage**: Helps in debugging, monitoring, and auditing the application's behavior.
  - **Log Levels:** Supports various levels of logging severity like INFO, WARNING, ERROR, etc., allowing filtering of log messages.
  - **Output:** Can direct log output to various destinations, such as the console, files, or network services.

- **`try-with-resources`:**
  - **Purpose:** To automatically manage resources (like files, sockets, streams) and ensure they are closed properly, *even if exceptions occur*.
  - **Automatic Closure:** Any resource declared within the parentheses of the `try` statement (e.g., `Socket`, `BufferedReader`, `PrintWriter`) will be automatically closed at the end of the `try` block, regardless of whether the block completes normally or throws an exception. This prevents resource leaks.

- **`IOException`:**
  - **Checked Exception:** `IOException` is a *checked exception*. This means that the Java compiler *forces* you to handle it, either by catching it with a `try-catch` block or by declaring that your method `throws IOException`.

- **Networking Errors:** Networking operations are prone to errors (network disconnections, timeouts, address resolution problems, etc.). `IOException` and its subclasses represent these types of errors.

- **`localhost`:**
  - **Loopback Address:** `localhost` is a special hostname that always refers to the *current* machine (the machine the code is running on). It's equivalent to the IP address 127.0.0.1 (the loopback address).
  - **Testing:** Using `localhost` is convenient for testing client-server applications on a single machine.